



Avaliação Bimestral 2 - Especificação e Implementação de endpoints de API WEB

Linguagem Técnica de Programação 3 - LTPI3

Prof. Luiz Gustavo Vêras

A atividade prática do 2º Bimestre irá corresponder à **10,0 pontos** e será realizada em duas etapas:

- ETAPA 1 - Programação e teste dos *models* em python criados a partir o modelo de entidades desenvolvidas na disciplina de PJI - **vale 1,5 ponto**;
- ETAPA 2 - Especificação e implementação de API WEB software do projeto integrador - **vale 8,5 pontos**;

O projeto deverá ser realizado em grupo (o mesmo do projeto integrador) ou individualmente. Sugere-se que cada etapa seja desenvolvida em sequência.

ETAPA 1 - Programação e teste e dos models (1,5 ponto / 10,0 de pontos)

Nesta etapa os grupos devem implementar em classes Python as entidades desenvolvidas no projeto de PJI. Essas classes serão posteriormente adicionadas à pasta */models* do **projeto Flask** e utilizadas para representar os dados da API Web.

Os */models* também serão importantes para realizar a integração com o banco de dados no próximo bimestre.

Os critérios de pontuação para esta etapa serão:

- Implementação coerente em Python das entidades definidas no PJI, com uso adequado de atributos e relacionamentos entre as classes **(0,5 ponto)**;
- Nomeação clara e padronizada de classes, atributos e métodos **(0,25 ponto)**;
- Encapsulamento e comportamento das classes, com uso adequado de construtor (`__init__`), método `__str__` e validações de regras de negócio nos métodos quando pertinente **(0,25 ponto)**;

- Desenvolvimento de script(s) de teste demonstrando o funcionamento correto das classes por meio da criação e manipulação de objetos **(0,25 ponto)**;

ETAPA 2 - Especificação e implementação de API WEB software do projeto integrador (8,5 pontos / 10,0 de pontos)

Nesta próxima etapa, os grupos devem especificar os endpoints de uma API WEB e implementá-las utilizando o *framework Flask* na linguagem Python. Utilizando como base o exemplo da Tabela 1 para uma **aplicação de registro de cadastro no SUS**, preencha as linhas com os *endpoints* que serão utilizados pela sua aplicação web.

Tabela 1 - Endpoints para uma aplicação fictícia do SUS.

Verbo HTTP	Descrição da Funcionalidade	Path	Path (Exemplo)	Dados de envio em JSON (Body)	Dados de Resposta em JSON (Body)
GET	Recupera a lista de cadastrados no SUS.	/api/sus/cadastro	GET /api/sus/cadastro	-	<pre> 1 - [2 - { 3 "nome": "Adoentado 1", 4 "idade": 30, 5 "codigo_saude": 123456789 6 }, 7 - { 8 "nome": "Adoentado 2", 9 "idade": 18, 10 "codigo_saude": 987654321 11 } 12] </pre>
GET	Recupera um cadastro no SUS pelo código de saúde.	/api/sus/cadastro/{codigo_saude}	GET /api/sus/cadastro/987654321		<pre> { "nome": "Adoentado 2", "idade": 18, "codigo_saude": 987654321 } </pre>
POST	Cria um novo cadastro.	/api/sus/cadastro	POST /api/sus/cadastro	<pre> { "nome": "Adoentado 3", "idade": 28, "codigo_saude": 99999999 } </pre>	-
PUT	Atualiza um cadastro da base do SUS pelo código de saúde.	/api/sus/cadastro/{codigo_saude}	PUT /api/sus/cadastro/987654321	<pre> { "nome": "Adoentado 2", "idade": 45, "codigo_saude": 987654321 }, </pre>	-
DELETE	Remove um cadastro da base do SUS pelo código de saúde..	/api/sus/cadastro/{codigo_saude}	DELETE /api/sus/cadastro/987654321	-	<pre> { "nome": "Adoentado 3", "idade": 45, "codigo_saude": 99999999 } </pre> *Retornou o objeto removido

Após especificar a API do seu projeto na tabela, é hora de implementá-la em um projeto Flask.

Abaixo segue uma indicação do que deve constar para guiar o aluno no desenvolvimento.

- Recupere as funcionalidades/requisitos funcionais planejadas do PJI e planeje os *endpoints* da API WEB para o seu projeto (Tabela de Verbos HTTP e Rotas) utilizando como base a Tabela 1;

- Devem ser criados **endpoints** que implementam CRUDs para pelo menos **duas entidades** do domínio/tema/model do PJI, os quais foram definidos na etapa anterior. Busque utilizar:
 - GET
 - GET com parâmetro
 - PUT com parâmetro
 - POST com parâmetro
 - DELETE com parâmetro
- Nos endpoints, os *models* e DAOs já devem ser utilizados;
- Realize os tratamentos de validações nos métodos os *endpoints* gerando as respostas HTTP específicos para cada situação;
- Cada grupo deve submeter no GitHub as implementações realizadas.

Assim os critérios de correção serão:

- Especificação correta dos endpoints e verbos HTTP **(1,5 ponto)**;
- Implementação funcional dos *endpoints* CRUD **(3,0 pontos)**;
- Organização e Integração adequada entre rotas, models e DAO **(1,5 ponto)**;
- Validações e tratamento de erros HTTP **(1,0 ponto)**;
- Organização, legibilidade e padronização do código da programação da API Flask conforme as aulas **(1,0 ponto)**;
- Estrutura e documentação do repositório GitHub **(0,5 ponto)**.

FORMA DE ENTREGA E APRESENTAÇÃO

- A tabela de especificação dos endpoints deverá ser adicionada ao repositório do projeto, podendo estar em formato PDF, PNG ou JPG, preferencialmente referenciada no arquivo **README.md**;
- O projeto Flask contendo as implementações da API WEB deverá ser disponibilizado em repositório no GitHub;
- Cada grupo deverá preparar uma apresentação em PDF contendo:
 - tema do projeto;
 - entidades implementadas;

- tabela de endpoints;
- principais funcionalidades da API;
- exemplos de requisições e respostas;
- A demonstração prática da API deverá ser realizada utilizando o **Thunder Client, Postman ou ferramenta equivalente**, evidenciando o funcionamento dos endpoints implementados;
- As correções serão realizadas considerando o último *commit* enviado no repositório até a data limite definida pelo professor;
- O link do repositório GitHub deverá ser enviado no Moodle e apresentado em sala de aula em **data e horário definidos pelo professor**.
- O tempo de apresentação será de até **20 minutos**.

Bom trabalho!